Table of Contents

S. No	Topic	Page No
1	Introduction to Python – Objectives, Scope & Applications	8–9
2	Python Environment & Fundamentals – Features, Installation & Interpreter	9–10
3	Basic Python Programming – Program Structure, Variables, Data Types & Operators	10–11
4	Input/Output & Control Statements – Conditions, Loops, break & continue	12–13
5	Pattern Programming	13
6	Functions, Collections & Strings – Functions, Lists, Tuples, Sets, Dictionaries	14–15
7	Searching, Sorting & Two-Dimensional Lists	15
8	Object-Oriented Programming – Classes, Objects, Inheritance, Polymorphism	16–17
9	Modules, Packages & Exception Handling	18
10	File Handling & Working with Data	19
11	Comprehensions, Iterators & Generators	19–20
12	Database Connectivity & CRUD Operations	20–21
13	Conclusion and Learning Outcomes	22

Introduction to Python

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its readable syntax allows programmers to express ideas with fewer lines of code and makes it suitable for beginners as well as professional developers.

❖ Objectives of the Training

➢ Core Understanding: Learn Python syntax, variables, data types, operators, and program structure.

➢ Problem Solving: Develop logical thinking through conditions, loops, functions, and simple algorithms.

➢ Practical Skills: Write, execute, test, and debug small Python programs.

❖ Scope of Python

➢ Web Development: Frameworks such as Django and Flask help build web applications.

➢ Data Science and Artificial Intelligence: Libraries such as NumPy, pandas, and scikit-learn support analysis and machine learning.

➢ Automation: Python can automate repetitive tasks, file operations, and reporting.

➢ Scientific Computing: Python is used in numerical calculations, simulations, and visualization.

❖ Importance of Python in Software Development

➢ Readable Syntax: Clear statements make code easier to understand and maintain.

➢ Large Ecosystem: A wide range of libraries supports diverse applications.

➢ Cross-Platform Support: Programs can run on Windows, macOS, and Linux with a suitable Python environment.

➢ Community Support: Documentation and community resources help programmers solve common problems.

Python Environment & Fundamentals

❖ Python Features

➢ Simple: Its syntax is concise and often resembles plain English.

➢ Interpreted: The Python interpreter executes program instructions.

➢ Dynamically Typed: The type of a variable is determined while the program runs.

➢ Object-Oriented: Python supports classes, objects, inheritance, and polymorphism.

➢ Portable: Python is available for major operating systems.

➢ Extensible: Python can work with modules and libraries written in other languages.

❖ Python Installation and Environment Setup

➢ Download the installer from the official Python website.

➢ Run the installer and enable the option to add Python to PATH when available.

➢ Verify the installation by running python --version or py --version in the terminal.

➢ Use IDLE, Visual Studio Code, PyCharm, or another compatible editor.

❖ Interpreter and Execution

Python source files generally use the .py extension. The interpreter runs the program and reports syntax errors or runtime exceptions. Interactive mode is useful for testing individual statements before placing them in a program.

❖ Basic Execution Flow

Source Code (.py) → Python Interpreter → Execution → Output

Basic Python Programming

❖ Basic Structure of a Python Program

A Python program can contain statements, variables, expressions, functions, and classes. Unlike some languages, Python uses indentation to define code blocks.

print("Hello, Python!")

❖ Variables and Data Types

➢ Variables are names that refer to values stored during program execution.

➢ int stores whole numbers; float stores decimal values; complex stores complex numbers.

➢ bool represents True or False, while str stores a sequence of characters.

➢ Common collection types include list, tuple, set, and dict.

name = "Python"\nage = 20\nprice = 12.5\nactive = True

❖ Type Conversion

Type conversion changes a value from one data type to another when appropriate.

number = int("25")\nvalue = float(10)\ntext = str(100)

❖ Operators

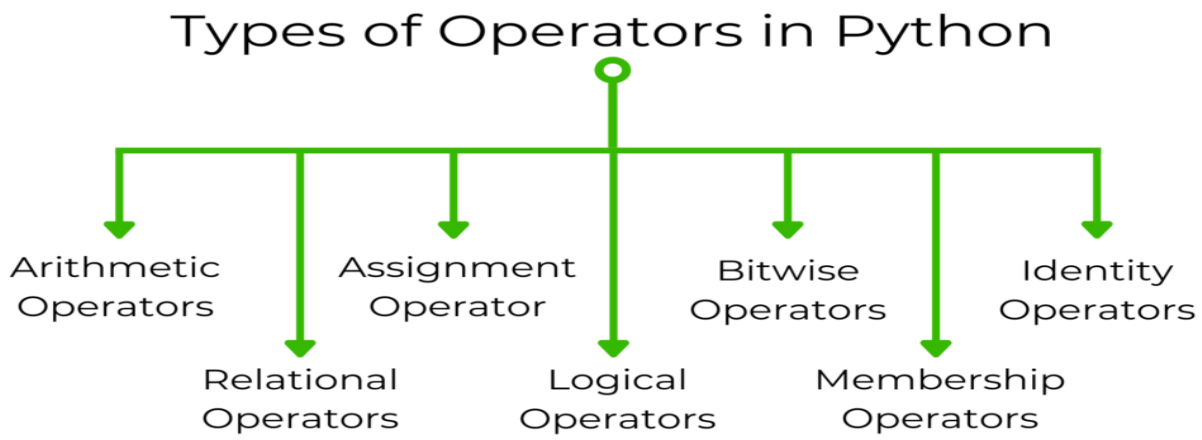
➢ Arithmetic: +, -, *, /, //, %, **.

➢ Comparison: ==, !=, >, <, >=, <=.

➢ Logical: and, or, not.

➢ Assignment: =, +=, -=, *=, /=.

➢ Membership and identity: in, not in, is, is not.



Input/Output & Control Statements

❖ Input and Output

The input() function accepts information from the user as text. The print() function displays values and messages on the screen.

name = input("Enter your name: ")\nprint("Welcome", name)

❖ Conditional Statements

➢ if executes a block when its condition is true.

➢ elif checks another condition when earlier conditions are false.

➢ else provides a default block when no previous condition matches.

marks = 75\nif marks >= 80:\n print("Grade A")\nelif marks >= 60:\n print("Grade B")\nelse:\n print("Keep practising")

❖ match Statement

The match statement compares a value against patterns and is useful when a program must select from several cases.

❖ Looping Statements

➢ for loop iterates over items in a sequence or another iterable.

➢ while loop repeats a block as long as its condition remains true.

➢ break terminates the loop, while continue skips to the next iteration.

for i in range(1, 4):\n print(i)

Control Statements and Pattern Programming

❖ Nested Loops

A nested loop is a loop placed inside another loop. It is useful for working with rows and columns, printing patterns, and processing two-dimensional data.

❖ Pattern Programming

Pattern programs use loops to display characters or numbers in a particular arrangement. They help beginners understand loop execution, counters, and nested repetition.

for i in range(3):\n for j in range(3):\n print("*", end=" ")\n print()

❖ Example Output

* *\n* * *\n* * *

❖ break and continue

➢ break immediately exits the nearest loop.

➢ continue skips the remaining statements in the current iteration and proceeds to the next iteration.

❖ Good Programming Practice

Use meaningful variable names, consistent indentation, and comments where they clarify the logic. Test programs with normal, boundary, and invalid inputs.

Functions, Collections & Strings

❖ Functions in Python

A function is a reusable block of code defined with the def keyword. It can accept parameters and return a result to the calling code.

def add_numbers(a, b):\n   return a + b\n\nprint(add_numbers(4, 6))

❖ Parameters and Return Values

➢ Positional arguments are matched by their position in the function call.

➢ Keyword arguments specify parameter names explicitly.

➢ Default arguments provide a value when an argument is omitted.

➢ return sends a result back to the caller.

❖ Strings

A string is an immutable sequence of characters. Common operations include indexing, slicing, searching, replacing, splitting, and changing letter case.

text = "Python"\nprint(text[0])\nprint(text.upper())

❖ Lists, Tuples, Sets and Dictionaries

➢ List: Ordered and mutable; duplicates are allowed.

➢ Tuple: Ordered and immutable.

➢ Set: Stores unique elements.

➢ Dictionary: Stores information as key-value pairs.

numbers = [10, 20, 30]\npoint = (2, 3)\nunique = {1, 2, 3}\nstudent = {"name": "Aman", "age": 20}

Collections, Searching & Sorting

❖ Common Collection Operations

➢ append() adds an item to a list; remove() deletes a matching item.

➢ len() returns the number of elements in a collection.

➢ Dictionary keys() and values() provide views of keys and values.

❖ Searching Algorithms

➢ Linear Search checks elements one by one until the target is found or the collection ends.

➢ Binary Search repeatedly halves a sorted collection to narrow down the target location.

❖ Sorting Algorithms

➢ Bubble Sort compares adjacent elements and swaps them when necessary.

➢ Selection Sort repeatedly selects the smallest remaining element.

➢ Insertion Sort inserts each element into its correct position in the sorted portion.

❖ Two-Dimensional Lists

A nested list can represent a table or matrix with rows and columns. The first index selects a row and the second index selects an item in that row.

matrix = [[1, 2], [3, 4]]\nprint(matrix[0][1]) # Output: 2

Object-Oriented Programming (OOP)

❖ Introduction to OOP

Object-oriented programming organizes software around classes and objects. It supports modularity, code reuse, and clear representation of real-world entities. Python supports object-oriented design alongside procedural and functional approaches.

❖ Classes and Objects

➢ Class: A blueprint defining attributes and behavior.

➢ Object: An instance created from a class.

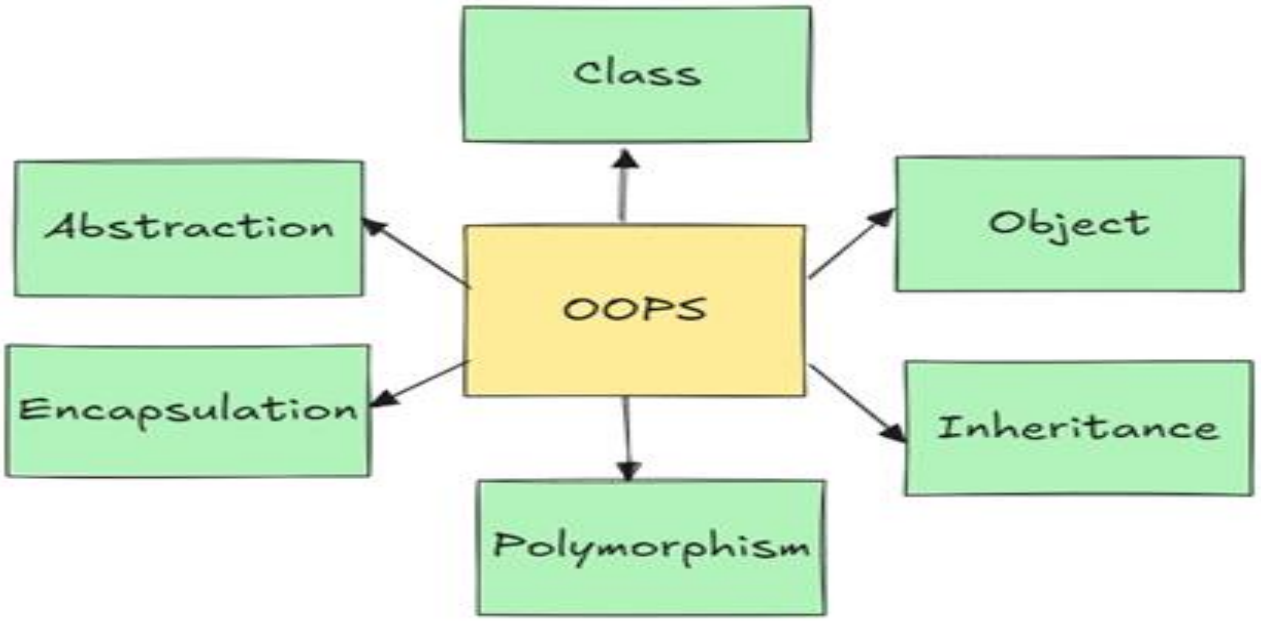
class Car:\n def __init__(self, model):\n self.model = model\n\ncar = Car("Hatchback")\nprint(car.model)

❖ Constructors

The __init__() method initializes an object's attributes when a new instance is created. The self parameter refers to the current object.

❖ Encapsulation

Encapsulation groups related data and methods inside a class. Naming conventions and properties can help manage access to internal state.



OOP Principles, Modules & Packages

❖ Inheritance

Inheritance allows a new class to reuse and extend attributes or methods of an existing class. It helps reduce duplication when classes share common behavior.

❖ Polymorphism

Polymorphism allows different object types to respond to the same operation in their own way. Python commonly supports this through method overriding and duck typing.

❖ Abstraction

Abstraction focuses on essential behavior while hiding implementation details. Abstract base classes can define methods that subclasses are expected to implement.

❖ Modules and Packages

➢ A module is a Python file containing reusable functions, classes, and variables.

➢ A package groups related modules into a structured namespace.

import math\nprint(math.sqrt(25))

❖ Useful Standard Modules

➢ math: Mathematical functions.

➢ random: Random numbers and selections.

➢ datetime: Date and time operations.

➢ os and pathlib: Operating-system and file-path operations.

Exception Handling

❖ Introduction to Exceptions

An exception is an event that interrupts normal program execution. Exception handling allows a program to respond to expected errors rather than terminating unexpectedly.

❖ try, except, else and finally

➢ try contains code that may raise an exception.

➢ except handles a matching exception.

➢ else runs when the try block completes without an exception.

➢ finally runs whether an exception occurs or not, making it useful for cleanup.

try:\n value = int(input("Enter a number: "))\n print(10 / value)\nexcept (ValueError, ZeroDivisionError):\n print("Enter a valid non-zero number.")\nfinally:\n print("Program finished.")

❖ Best Practices

➢ Catch specific exceptions instead of hiding every error.

➢ Use clear messages that help users correct invalid input.

➢ Avoid using exceptions as a replacement for ordinary program logic.

File Handling & Working with Data

❖ File Handling

Python uses open() to read and write files. The with statement manages the file resource and closes it automatically when the block finishes.

with open("notes.txt", "w", encoding="utf-8") as file:\n file.write("Learning Python")\n\nwith open("notes.txt", "r", encoding="utf-8") as file:\n print(file.read())

❖ File Modes

➢ r: Read an existing file.

➢ w: Write and replace file contents.

➢ a: Append content to the end of a file.

➢ x: Create a new file and fail if it already exists.

❖ Working with Structured Data

The csv module supports tabular text files, while the json module converts supported Python objects to and from JSON text. These formats are commonly used to exchange data between applications.

import json\ndata = {"language": "Python", "level": "Beginner"}\ntext = json.dumps(data)\nprint(text)

Comprehensions, Iterators & Generators

❖ List Comprehensions

A list comprehension creates a list from an iterable in a compact form. It can transform each item and optionally filter items using a condition.

squares = [n * n for n in range(1, 6)]\nprint(squares)

❖ Iterators

An iterator produces items one at a time. The iter() function obtains an iterator and next() requests the next item.

❖ Generators

A generator produces values lazily using yield. This can reduce memory use when a program processes a large sequence of values.

def count_up_to(limit):\n n = 1\n while n <= limit:\n yield n\n n += 1

❖ Applications

➢ Comprehensions make simple transformations concise.

➢ Iterators provide a standard way to traverse collections.

➢ Generators are useful for streams, large files, and computed sequences.

Database Connectivity

❖ Introduction to Database Connectivity

Python applications can connect to databases through appropriate drivers and libraries. SQLite is supported by the standard sqlite3 module and is suitable for small applications, demonstrations, and local storage.

❖ Database Workflow

➢ Import the required database module.

➢ Open a connection to the database.

➢ Create a cursor and execute SQL statements.

➢ Commit changes when needed and close resources.

❖ CRUD Operations

➢ Create: INSERT adds a new record.

➢ Read: SELECT retrieves records.

➢ Update: UPDATE modifies existing records.

➢ Delete: DELETE removes records.

❖ Example Database Setup

import sqlite3\nconnection = sqlite3.connect("students.db")\ncursor = connection.cursor()\ncursor.execute("CREATE TABLE IF NOT EXISTS students (id INTEGER, name TEXT)")\ncursor.execute("INSERT INTO students VALUES (?, ?)", (1, "Aman"))\nconnection.commit()

Database Operations & Conclusion

❖ Reading and Closing a Database

cursor.execute("SELECT*FROM
students")\nprint(cursor.fetchall())\nconnection.close()

❖ Good Database Practices

➢ Use parameterized queries to reduce SQL injection risks.

➢ Handle database errors and close connections reliably.

➢ Validate input before saving it to a database.

❖ Conclusion

This report introduces the core building blocks of Python programming, from basic syntax to functions, collections, file operations, exceptions, object-oriented design, and introductory database connectivity. Practising these topics helps learners develop structured problem-solving skills and write programs that are easier to read, test, and maintain.

❖ Learning Outcomes

➢ Write and execute basic Python programs.

➢ Use variables, operators, conditions, and loops to implement logic.

➢ Organize reusable code with functions, modules, and classes.

➢ Work with strings, collections, and files.

➢ Recognize and handle common runtime errors.

➢ Understand the basic workflow for storing and retrieving data.

❖ Future Learning

After mastering these fundamentals, learners can explore web development, data analysis, automation, machine learning, APIs, and software testing according to their interests.

Python Functions in Detail

Functions divide a program into smaller, reusable units. They make a program easier to test, understand, and maintain because a task can be written once and called whenever it is needed. A well-designed function normally performs one clear job and uses meaningful parameter names.

❖ Defining and Calling Functions

A function is defined with the def keyword, followed by a function name, parentheses, and a colon. The indented statements form the function body. The function executes when it is called by name.

def greet():\n print('Welcome to Python')\n\ngreet()

Function names usually use lowercase words separated by underscores. Parameters receive values from the caller, while return sends a result back. A function that reaches the end without a return statement returns None.

❖ Arguments and Parameters

➢ Positional arguments are assigned according to their position in the call.

➢ Keyword arguments identify the parameter by name and can make calls easier to read.

➢ Default parameters provide a fallback value when the caller omits an argument.

➢ Variable-length arguments using *args collect extra positional arguments; **kwargs collects extra keyword arguments.

Parameters should be chosen carefully. Avoid changing global variables when a result can be returned explicitly, because explicit inputs and outputs make functions easier to test.

❖ Local and Global Scope

A variable created inside a function is normally local to that function. A variable defined outside functions belongs to a wider scope. The global keyword can modify a module-level variable, but excessive global state makes programs harder to reason about.

Python resolves names using a scope search commonly described as Local, Enclosing, Global, and Built-in (LEGB). Understanding scope helps prevent unexpected name conflicts.

❖ Lambda Functions and Recursion

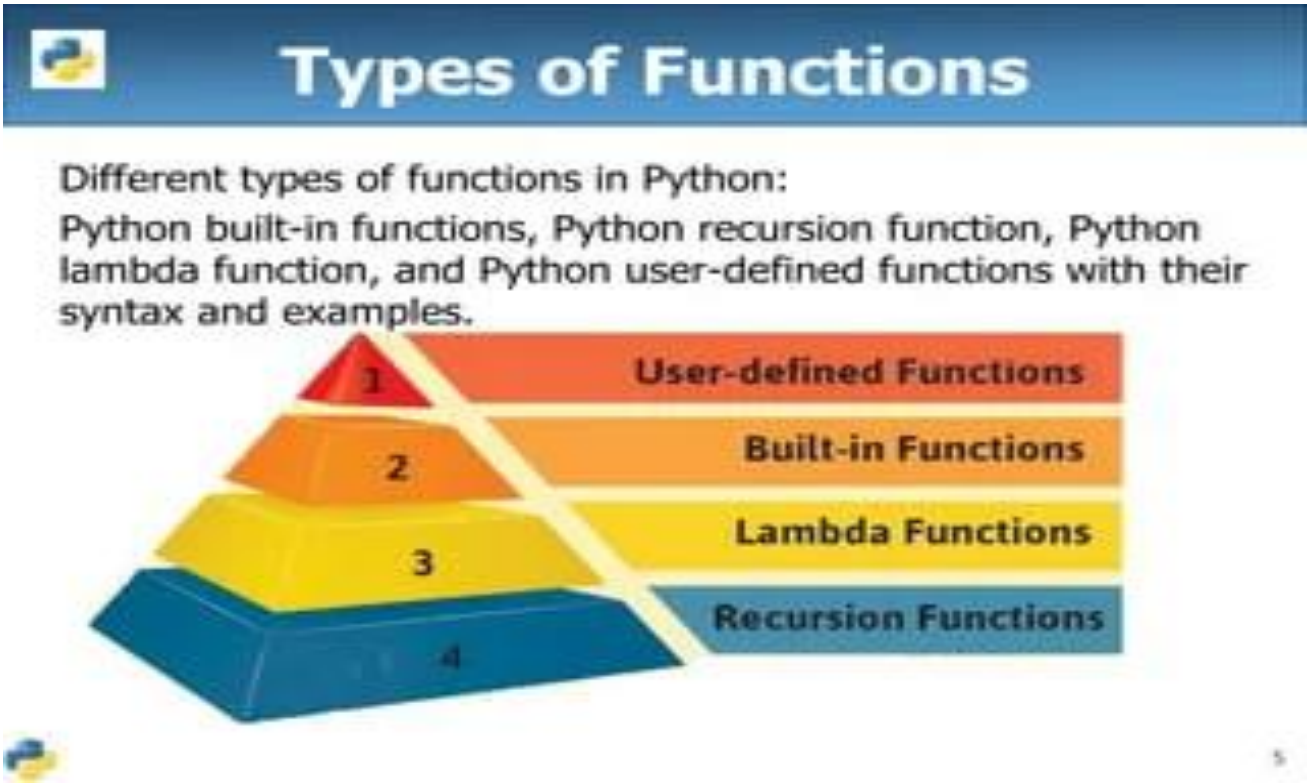
A lambda expression creates a small anonymous function containing one expression. It can be useful as a short key function for sorting, but ordinary def functions are usually clearer for multi-step logic.

double = lambda number: number * 2\nprint(double(5))

Recursion occurs when a function calls itself. A recursive function needs a base case that stops further calls. Without a correct base case, recursion can continue until Python raises a recursion-related error.

❖ Practical Example

def calculate_total(price, quantity=1):\n return price * quantity\n\nprint(calculate_total(50, 3))



Working with Strings

Strings are used to store names, messages, identifiers, file paths, and other text. In Python, a string is an immutable sequence of Unicode characters. Once created, a string cannot be changed in place; operations such as replacement create a new string.

❖ Creating and Accessing Strings

Strings can be written using single quotes, double quotes, or triple quotes for multi-line text. Indexing starts at zero, and negative indexes count backward from the end.

language = 'Python'\nprint(language[0])\nprint(language[-1])

Slicing selects a range with the form sequence[start:stop:step]. The stop index is excluded. Slicing is useful for extracting a prefix, suffix, or selected characters without changing the original string.

❖ Common String Methods

➢ lower() and upper() convert letters to lowercase or uppercase.

➢ strip() removes leading and trailing whitespace.

➢ replace(old, new) creates a string with matching text replaced.

➢ split() separates text into a list using a separator.

➢ join() combines strings from an iterable using a chosen separator.

➢ find() returns the position of a substring or -1 if it is not found.

String methods are especially useful for cleaning user input. For example, strip() can remove accidental spaces around a name, while lower() can normalize a response before comparing it.

❖ Formatting Strings

Formatted strings make it easier to place values inside text. An f-string begins with f before the opening quote and places expressions inside curly braces.

name = 'Aman'\nmarks = 82\nprint(f'{name} scored {marks} marks.')

Formatting can control decimal precision, alignment, and width. These features are helpful when displaying tables or reports.

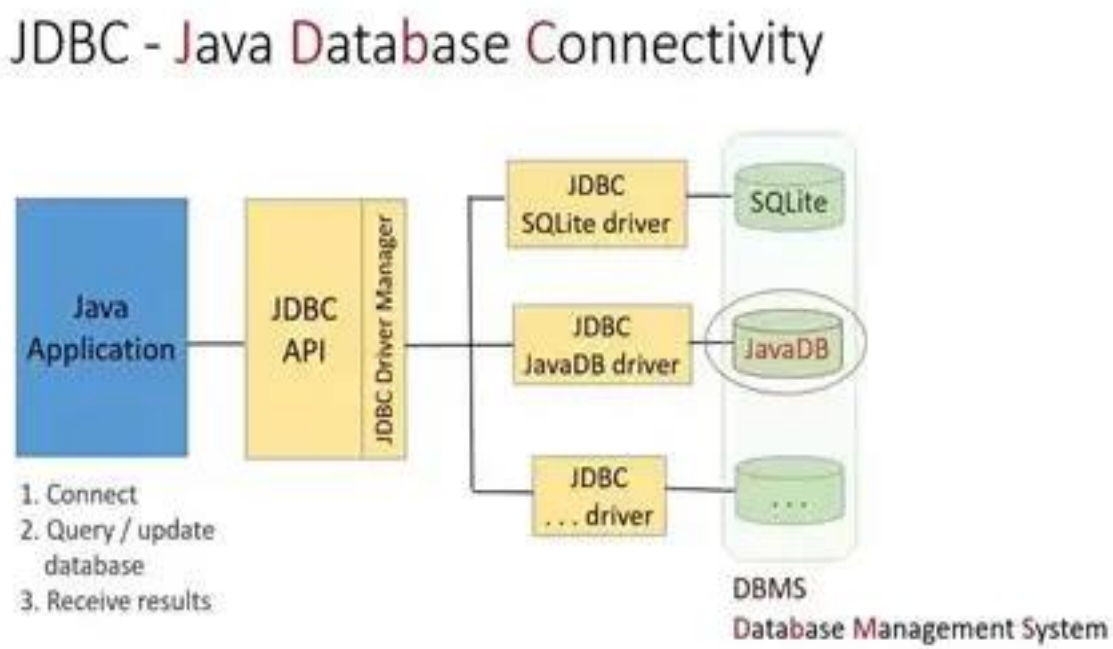
❖ Comparing and Validating Text

String comparisons are case-sensitive unless the text is normalized first. Input validation may check whether a string is empty, contains digits, or matches a required pattern. For more complex patterns, the re module provides regular expressions.

Validation should provide clear feedback and should not assume that every user enters data in the expected format.

❖ Practical Example

email = input('Enter email: ').strip().lower()\nprint('Entered:', email)



Lists and List Operations

A list is an ordered, mutable collection. It can store values of different types, although many programs use lists containing similar items. Lists are widely used for processing marks, names, measurements, and results returned from functions.

❖ Creating and Accessing Lists

A list is created with square brackets or the list() constructor. Items are accessed with indexes and ranges can be selected using slicing.

marks = [72, 81, 66, 90]\nprint(marks[0])\nprint(marks[1:3])

Lists can contain other lists, making it possible to represent tables or matrices. The length of a list is returned by len(). An empty list is written as [].

❖ Adding, Updating and Removing Items

➢ append(item) adds one item at the end.

➢ extend(iterable) adds all items from another iterable.

➢ insert(index, item) inserts an item at a chosen position.

➢ remove(item) removes the first matching value.

➢ pop(index) removes and returns an item; without an index, it removes the last item.

➢ sort() sorts the list in place; sorted(list) returns a new sorted list.

Before removing an item, a program should consider whether it exists. This avoids errors and makes behavior predictable when user-provided values are involved.

❖ Traversing Lists

A for loop is the usual way to visit each list item. enumerate() can provide both the index and the value. A while loop can also be used when the program needs explicit control over the index.

subjects = ['Python', 'Math', 'English']\nfor index, subject in enumerate(subjects, start=1):\n print(index, subject)

List comprehensions offer a compact way to build a list from an existing iterable. They are most readable when the transformation and condition are simple.

❖ Copying and Aliasing

Assigning one list variable to another does not copy the list; both names refer to the same object. Use list.copy() or slicing to create a shallow copy. Nested mutable items may still be shared by a shallow copy.

This distinction matters when modifying collections inside functions or when storing an original version before applying changes.

❖ Practical Example

numbers = [4, 1, 9, 2]\nnumbers.sort()\nprint(numbers) # [1, 2, 4, 9]

❖ Practice Questions

➢ Accept five numbers and print their sum.

➢ Find the largest and smallest values in a list.

➢ Remove duplicate values from a list.

➢ Sort a list in ascending and descending order.

Tuples, Sets and Dictionaries

Python provides several built-in collection types for different purposes. Choosing the right collection improves clarity and can make operations more efficient. Lists suit changing sequences, tuples suit fixed groupings, sets represent unique values, and dictionaries map keys to values.

❖ Tuples

A tuple is ordered and immutable. It is often used to group values that belong together, such as coordinates or a database record. A one-item tuple requires a trailing comma.

point = (10, 20)\nsingle_item = ('Python',)\nprint(point[0])

Tuple unpacking assigns tuple elements to multiple variables in one statement. The number of target variables should match the number of values unless starred unpacking is used.

❖ Sets

A set stores unique hashable elements and is useful for removing duplicates and performing mathematical set operations. Sets do not provide positional indexing.

a = {1, 2, 3}\nb = {3, 4, 5}\nprint(a | b)\nprint(a & b)

➢ Union combines elements from both sets.

➢ Intersection returns common elements.

➢ Difference returns elements in one set but not the other.

➢ Membership testing with in can be useful when checking whether a value is present.

❖ Dictionaries

A dictionary stores key-value pairs. Keys must be hashable and are commonly strings or numbers. Dictionaries preserve insertion order in modern Python and are useful for structured records.

student = {'name': 'Aman', 'marks': 85}\nprint(student['name'])\nstudent['marks'] = 90

The get() method can retrieve a key without raising KeyError when the key is absent. The items() method provides key-value pairs for iteration.

Algorithmic Thinking and Debugging

Programming is not only about writing syntax; it also requires planning a solution, testing assumptions, and identifying errors. Algorithmic thinking breaks a problem into smaller steps that can be implemented and verified.

❖ Understanding a Problem

First identify the inputs, required outputs, constraints, and special cases. For example, a program that calculates an average must decide what should happen when the input list is empty.

Write the solution in plain language or pseudocode before coding. This helps separate the logic from the details of Python syntax.

❖ Testing and Debugging

➢ Syntax errors occur when code violates Python grammar, such as a missing colon.

➢ Runtime errors occur while a program executes, such as dividing by zero.

➢ Logic errors occur when the program runs but produces an incorrect result.

Test normal inputs, boundary values, and invalid inputs. Use short test cases that make it easy to understand why an output is expected.

❖ Readable and Maintainable Code

Use meaningful names, consistent indentation, and small functions. Comments should explain intent or non-obvious decisions rather than repeat every line of code.

When a bug occurs, reproduce it reliably, isolate the smallest section that causes it, inspect values, and change one thing at a time. This method is usually more effective than making several unverified changes.

❖ Basic Complexity Awareness

Time complexity describes how the amount of work grows as input size increases. Linear search may inspect every element, while binary search can narrow a sorted list quickly by halving the remaining search range.

Space complexity considers additional memory used by an algorithm. These ideas help learners compare solutions beyond whether they produce the correct output.

Python Applications and Next Steps

Python fundamentals provide a base for more specialized areas. The best next step depends on the learner's interests and the type of work they want to build. A small portfolio of tested programs can demonstrate understanding more clearly than a list of topics alone.

❖ Web Development

Web frameworks such as Django and Flask help developers create websites and backend services. Learners should understand HTTP basics, routing, templates, forms, authentication, and safe handling of user input before building larger applications.

❖ Data Analysis and Visualization

The pandas library supports tabular data analysis, while NumPy provides array-oriented numerical operations. Visualization libraries can help communicate patterns, trends, and comparisons. Before using advanced libraries, learners should be comfortable with lists, dictionaries, functions, and file formats such as CSV.

❖ Automation and Scripting

Python scripts can organize files, rename batches of documents, process text, create reports, and automate repetitive tasks. Reliable automation should handle missing files, invalid inputs, and unexpected errors rather than assuming every step will succeed.

❖ Machine Learning and Artificial Intelligence

Machine learning projects typically involve preparing data, choosing a model, training it, evaluating results, and communicating limitations. A foundation in programming, statistics, and data handling is useful before moving to machine learning libraries.

❖ Suggested Practice Projects

➢ Student marks manager: store marks, calculate averages, and assign grades.

➢ Expense tracker: record categories, dates, and amounts, then summarize spending.

➢ File organizer: sort files into folders by extension while handling name conflicts.

➢ Quiz application: ask questions, validate answers, and calculate a final score.

➢ Contact book: store, search, update, and delete contact records.

❖ Conclusion

Consistent practice turns syntax knowledge into programming skill. By revisiting fundamentals, testing programs carefully, and completing small projects, learners can develop confidence and prepare for further study in web development, data analysis, automation, or machine learning.